Synthetic Data Generation for Sounding Object Detection

Nael Belhaj Haddou (s4431278)

Supervisors:
Hazel Doughty & Rita Pucci

Master Thesis– Computer Science

Leiden Institute of Advanced Computer Science (LIACS)
liacs.leidenuniv.nl

June 28, 2026

Abstract

Segmenting sound sources in collision videos is difficult in large part because of the variety of possible collision sounds and sources. However, the availability of labeled video data is very limited compared to what model training requires. We introduce a synthetic data generation and labeling pipeline for collision videos. Unity is chosen to minimize simulation overhead during data generation, and SAM2 is used in labeling for its versatility and video processing features. We demonstrate that the generated data alone is close enough to real data in its distribution to enable some improvements that partially generalize to a real dataset, briefly improving validation accuracy from 38% to 43% but quickly overfitting the synthetic data. However, synthetic data alone does not match real-data-only training, and using it as a warmup or supplement to real data provides no additional benefit. Lastly, we find that within the scales studied ($\sim$10,000 samples) data quantity is not the limiting factor. At this dataset scale, label quality and the sim2real gap appear to bound performance more than dataset size.

Contents

1 Introduction

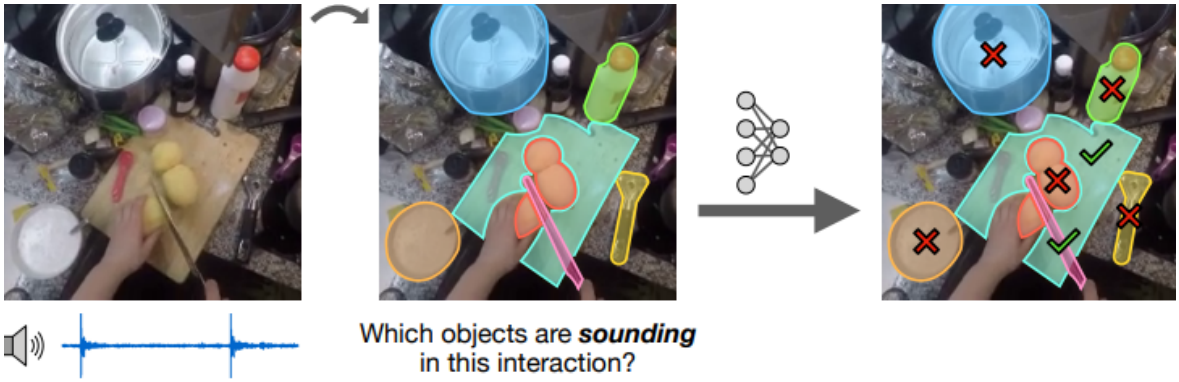


Figure 1: An illustration of the sounding object detection task at evaluation. We start with an audio and a visual modality. We then segment all objects in the visual modality. Finally we use the model to select the masks that correspond to the sounding objects in the interaction. Reproduced from Yang et al. [15] (CC BY 4.0).

1.1 The situation

Identifying the source of simple sounds in videos is a trivial task for humans and has seen very effective research [12, 9]. However, despite their frequency, collision sounds and identification of their source is far less explored. This goal presents a complexity that simple sounds do not, the diversity of interaction sounds. In a collision the sound depends on both objects and on many further factors, so even the natural label space is unclear: a target could be a specific object, a coarse material pair such as a wood–plastic impact, or the action itself. This ambiguity is what makes collision sounds harder to model than the isolated or ambient sounds studied so far.

Recent work has begun to tackle these interaction sounds. The task we study is identifying which object in a video produces a given sound, or sounding object detection. This task was introduced by Yang et al. [15], building on the sounding action discovery of Chen et al. [2]. We adopt their task and model as our starting point and baseline.

Progress on this task is limited by data rather than model capacity. Labeled collision videos are scarce because they must be collected and annotated by hand, far more slowly and expensively than the resulting data is consumed in training. Existing collision datasets inherit this cost, and with it a fixed, uncontrolled distribution of collision types and materials. We therefore ask whether collision training data can instead be produced and labeled in a cheap and controllable fashion using simulation and video segmentation models.

We address the following research question: Can synthetic collision data improve sounding object detection performance, either by training on it directly or by using it alongside real data? We also investigate whether the improvement bottleneck for this synthetic data is quantity or quality.

In summary, the main contributions of this thesis are:

- A synthetic data generation and labeling pipeline for collision videos, combining Unity and SAM2.
- An empirical evaluation of strategies for combining synthetic and real data, showing that synthetic data alone enables meaningful learning but provides no additional benefit when real data is available.
- Evidence that, within the scales studied, increasing synthetic quantity does not raise performance, pointing to label quality and the sim2real gap as the more likely limiting factors.

1.2 Thesis overview

The remainder of this thesis is organized as follows. Section 2 discusses works related to our approach in the fields of sounding object detection, simulation and synthetic audio data generation. Section 3 explains the method we use throughout this work. Section 4 presents the experimental setup, results and their interpretation. Section 5 discusses our key contributions, limitations of our method and potential avenues for future improvements.

2 Related Work

2.1 Sound Source Segmentation

Localizing the source of a sound in a video is a long-standing audio-visual problem, with work learning to point to sounding regions in a scene [12, 9]. These methods target where a sound originates rather than which object produces it, and are typically evaluated on isolated or ambient sources rather than collisions. Closest to our setting is recent work on interaction and collision sounds. Yang et al. [15] introduced sounding object detection together with CCT, the model we build on (Section 3), reaching state of the art by pairing an object-centric encoder with the action-discovery training of Chen et al. [2]. Most directly related to this thesis is the concurrent work of Parida et al. [10], which segments collision sound sources in egocentric video. As with CCT, it relies on real footage for both training and evaluation.

This shared dependence on natural egocentric footage introduces two structural constraints. First, real-world collection is expensive in both capture and annotation effort, and the scenarios covered are dictated by what cameras happen to observe rather than by experimental design. Second, audio-visual co-occurrence in real footage is often confounded by ambient noise, making it difficult to control the precise acoustic properties of individual contact events. Together these bound the diversity of collision types and surface materials a model can be exposed to during training. We address this bottleneck by asking whether physically grounded synthetic data can substitute for or augment real footage, decoupling training-set composition from the constraints of in-the-wild recording. Such a pipeline needs two ingredients: a simulation backend to cheaply generate collision scenes, and a sound engine to give them physically faithful audio. We review each in turn.

2.2 Physics-based simulation for perception

Physics-based simulation has a long history as a data generation strategy in machine learning, but the introduction of modern game engines as general-purpose simulation backends (Unity, Unreal, etc) represents a qualitative shift in accessibility and fidelity. Juliani et al. [7] formalized this perspective, proposing a taxonomy in which engines such as Unity serve as flexible scaffolding for generating controlled training environments, reducing engineering overhead compared to custom simulators while retaining expressive physical modelling. These advantages were strengthened empirically when Borkman et al. introduced a synthetic data package for Unity [1] that allowed them to train a model with synthetic data and outperform another model trained on real data, successfully physically grounding their model and bridging the sim2real domain gap. The sim2real domain gap is the difference in data distribution between reality and any given simulation which often limits the usefulness of synthetic data.

ThreeDWorld [6] by Gan et al. extends this paradigm by providing a rich simulation backend. It couples rigid body physics, audio and support for mobile agents making it extremely versatile. While this breadth is a notable strength, it comes with more significant configuration complexity that represents unnecessary overhead for narrowly scoped tasks such as collision sound segmentation. Unity, by contrast, offers a more lightweight deployment path while still exposing the physics and audio subsystems needed to simulate contact acoustics. This trade-off motivates our choice of Unity as the simulation backbone for synthetic data generation in this work. Simulation supplies the scene and rigid-body physics, but the sounding object detection task requires audio that accurately reflects the mass, velocity, material and other parameters not handled by a game engine's audio systems. We therefore turn to synthetic audio.

2.3 Synthetic Data in Audio Learning

The use of synthetic data in audio-driven learning is a comparatively nascent area relative to its visual counterpart, yet recent work has begun to close this gap. In 2025, Cherep et al. [3] proposed an audio synthesis pipeline for constructing large-scale datasets that span the diversity of real-world sounds. Their central contribution is a method for generating minimally contrastive pairs: sounds that differ along a single controllable acoustic dimension, making them ideal negative pairs for contrastive objectives without relying on augmentation artifacts. Using this approach, they surpass strong real-data baselines such as CLAP [5] on several benchmarks, demonstrating that synthetic audio can serve as a credible substitute rather than a mere proxy.

Critically, however, the approach produces sounds through stochastic synthesis rather than physical simulation, meaning the resulting audio lacks a grounded correspondence to the physical properties of objects, namely material, mass, and contact geometry, that govern real collision acoustics. This is a fundamental limitation for our setting. In 2019, Traer et al. [13] offered a principled alternative grounded in both physics and auditory perception. Rather than computing high-resolution solutions, which are too slow for scalable data generation, they demonstrate that the auditory system infers material categories from the statistics of resonant modes rather than their precise spectral configuration. This observation motivates a source-filter synthesis model in which object impulse responses are generated by sampling modal parameters from empirical distributions fit to real impact recordings, conditioned on coarse material labels, and then convolved with a spring-model contact force parameterized by mass and velocity. Perceptual validation confirms

that listeners cannot reliably distinguish synthetic impacts from real recordings. Crucially, the only inputs required at synthesis time are quantities already tracked by physics engines. Our work exploits this directly: we use Clatter, a Unity plugin implementing the Traer et al. model, to generate physically grounded audio tied to simulated object properties, offering a physically interpretable source of variation that aligns directly with the multi-modal structure of the sounding object detection task.

3 Approach

The task studied here is sounding object detection as defined by Yang et al. [15]. Here a sounding object is one that is directly part of an interaction generating sound. Formally, our model is given three inputs: video $V \in \mathbb{R}^{T\times W\times H\times 3}$, audio $A \in \mathbb{R}^{S}$ and narration L (although language isn't used here). During inference, we use a pool of candidate object masks. The model then computes a similarity map between image patches and the audio embedding and scores are finally pooled across objects using overlap between candidate masks and patches, resulting in the per-object score. Unlike other audiovisual localization, this task focuses on which objects are part of the interaction rather than where the sound's source is.

Here we build a simulation to labeled data pipeline for the two modalities we use: video and audio. An overview of its parts (red) and the data they exchange (cyan) is available in diagram 1.

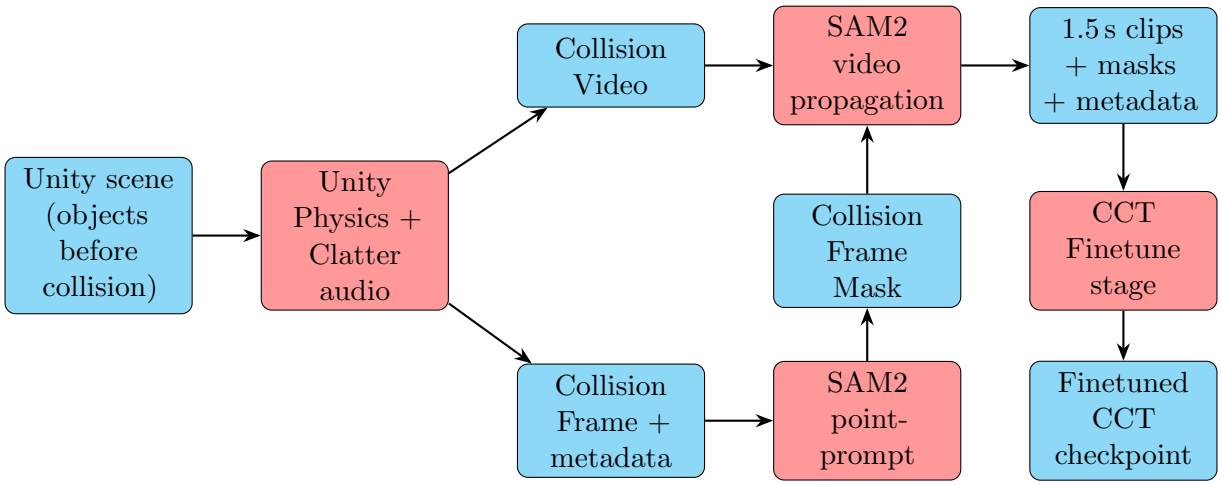


Diagram 1: A diagram of the data generation, labeling and training pipeline used in this thesis. First the scene with the objects goes through the simulated collisions using the Unity and Clatter backends. Then the resulting video and metadata are passed to SAM2 which segments the objects using the point prompt before propagating the masks to other frames. SAM2 then outputs the clips, masks and metadata. The clips allow the finetune stage of training to distinguish background parts of the clip and enforce the object-centric prior.

3.1 Data Generation

To generate coherent collision data a physics informed sound generation method is necessary. Here the method outlined by Traer et al. [13] is used through the clatter plugin for unity. This method is based on the popular source-filter model which uses the contact force of a collision and the impulse response of the objects involved to deduce the sound to produce. To make this more tractable the transient component of the impulse response is approximated by a "click" on multiple broad frequency bands that fade at different rates. The part of the sound that resonates after the initial impact sound is modeled using decaying sine waves at different resonant modes of the modeled object.

As mentioned earlier we generate the dataset using the Unity engine for its little overhead for a simple simulation. For each clip to be generated, a set number of random objects in the project are set at random points on a sphere and thrown to the center. Here we use 6 objects as we find empirically that this is a good middle ground between collisions per video and visual clutter for our uses here. At the same time, the camera is placed at a random (within bounds) distance and angle from the center of the sphere which helps get various angles to improve data diversity. The reason for such an initialization is that a sphere has all its surface points at an equal distance from its center. This means that given the same initial speed towards the center of the sphere, objects will most likely collide at the center. As a consequence, by pointing the camera at the center of the sphere we know that we will capture the majority of collisions happening without having to make the angles of the collisions deterministic. This improves data diversity without increasing object diversity as that allows for different video and sound modalities to be generated from the same set of objects.

For each collision detected, metadata (object visibility, collision location, etc.) is collected for use in labeling. After 5 seconds the recording ends and the objects are reset for a possible next video. This length ensures all possible collisions with this configuration have happened. The videos along with their metadata are then passed to a script for labeling.

3.2 Labeling

The labeling script iterates over every collision of each video. We use the SAM2 [11] model to label the clips. We choose SAM2 for multiple reasons. The first is to improve comparability with CCT as SAM2 is the model used by Yang et al. [15] to create the segmentation masks they use to train the model on their real datasets. The second is that SAM2 has impressive zero-shot generalization capabilities compared to closed-vocabulary models trained on smaller datasets like YOLO. We know this to be important here because there will necessarily be a gap between any segmentation model's training data and the clips we use, meaning generalization capabilities will be important for this application. Specifically, we use the base-size SAM 2.1 checkpoint, which balances accuracy and cost. Version 2.1 was chosen for its improvements over SAM 2 on visually similar objects, small objects, and tracking occluded objects, all relevant advantages here. The labeling script was run on an NVIDIA GTX Titan X. As we use an external model to label the data, some noise in the labels is inevitable which makes the training method studied here weakly supervised.

First, it pulls the collision frame and uses SAM2 to get a mask of the two colliding objects

from the point prompt included in the metadata. We use a point prompt on the collision frame to ensure the colliding objects are selected, since during clip creation we verified both colliding objects had their centers visible to the camera and the points of the prompt are the centers of these same colliding objects. The mask is then propagated to the next frames using SAM2's video feature until the clip is up to 0.5s long after the collision. The video is then reversed and the propagation process is repeated to label up to 1s before the collision. Putting these two parts together gives a 1.5s clip and corresponding mask in the same format as in [15]. Figure 2 shows some frames from a correctly labeled collision video between a metal sphere and a wooden barrel.

Figure 2: Some frame examples of a correctly labeled synthetic video in chronological order. The objects involved in the collision labeled here are one of the metal balls and the wooden barrel. Masks follow the objects and cover them entirely without bleeding onto other nearby objects.

3.3 Training

The model here uses a slot attention encoder as introduced by Locatello et al. [8]. This encoder takes N input vectors and learns object-focused representations output as K slots. Slots are initially assigned to random parts of the input and "compete" every iteration to explain parts of the input before being updated using a learned recurrent update function such as a Gated Recurrent Unit (GRU).

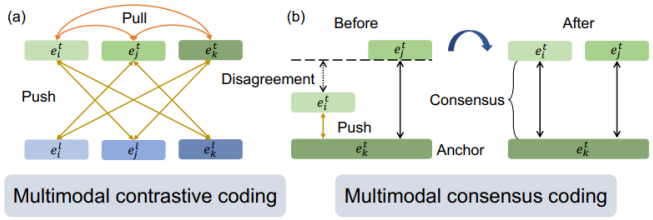


Figure 3: An illustration of Multimodal Contrastive Consensus Coding (MC3). (a) In the align stage we take the 3 modalities from a given sample and nudge them closer together so they agree. At the same time we make the modalities from another sample get further away. (b) Once we get to the refine stage modalities from a given sample are closer to each other than other samples but may not agree on how close they should be. This is why we push them away from the anchor modality for this sample until they both are at the same distance from the anchor. Reproduced from Chen et al. [2] (CC0).

Multimodal Contrastive Consensus Coding Training The multimodal contrastive consensus coding (MC3) loss was introduced by Chen et al. [2]. It involves a two stage model

training for the sounding action discovery task introduced in the same paper. This task involves having a model distinguish whether a human action in a video with a caption is sounding, whether it causes sound by interacting with the surrounding environment. The training starts with the embeddings from our encoder for each data modality: $e_l \in \mathbb{R}^D$ for language, for visual $e_v^* \in \mathbb{R}^{T \times N \times D}$ and for audio $e_a \in \mathbb{R}^D$ with D as the common embedding dimension, T the number of input video frames and N the number of non-overlapping patches. The video embedding is not used directly, first the object mask is patchified into $\overline{M} \in \mathbb{R}^{T \times N \times 1}$. This patchified object mask is then applied to the original video embedding e_v^* and mean pooled over non-zero features across T and N to obtain the visual embedding vector $e_v \in \mathbb{R}^D$ used in training.

The align stage aims to make pairs of modality embeddings group together based on the interaction they captured. To this end, we train with the InfoNCE loss [14] function, a contrastive loss. The loss $\mathcal{L}_{x \to y}$ is outlined in equation 1 where x and y are two modalities, e_x^i is the embedding of modality x for the ith sample, τ is the temperature and $\mathcal{B}$ is the batch size

$$\mathcal{L}_{x \to y} = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \log \frac{\exp(e_x^i e_y^i / \tau)}{\sum_{l \in \mathcal{B}} \exp(e_x^i e_y^l / \tau)} \tag{1}$$

$$\mathcal{L}_{y \to x} = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \log \frac{\exp(e_y^i e_x^i / \tau)}{\sum_{l \in \mathcal{B}} \exp(e_y^i e_x^l / \tau)} \tag{2}$$

The opposite pair has loss as outlined in equation 2. The total align stage loss is the sum of these symmetric pairs over every modality. This takes the form: $\mathcal{L}_{align} = \sum_{x,y} \mathcal{L}_{x \to y} + \mathcal{L}_{y \to x}$.

During the refine step modality similarity scores from a given video are made to agree. This is done using an anchor modality A to which all other modalities are compared (following [2]). A consensus term then pulls embeddings from a given sample closer.

Finetuning In CCT [15], Yang et al. extend the MC3 training with a third finetune stage and introduce the sounding object detection task. This new finetuning stage aims to penalize the model for attending to background patches, those that are unrelated to the sounding object mask M. This is to induce localized understanding in the model which isn't very important for sounding action discovery but is for sounding object detection.

The loss used in this stage is InfoNCE like in the align step. The main difference is that here we also use negative pairs from background region embeddings of the same sample along the pairs from the other samples' embedding e_v. One side of the loss can be written as $\hat{\mathcal{L}}_{a \to v}$ as shown in eq 3 where $\hat{e}_v^m$ is the visual embedding of a random sample of half the non-object regions from video m. Here, we define background or non-object regions as the regions not related to the patchified object mask.

$$\hat{\mathcal{L}}_{a \to v} = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \log \frac{\exp(e_a^i e_v^i / \tau)}{\sum_{l \in \mathcal{B}} \exp(e_a^i e_v^l / \tau) + \sum_{m \in \mathcal{B}} \exp(e_a^i \hat{e}_v^m / \tau)} \tag{3}$$

We can now define the full loss of the finetune stage as $\mathcal{L}_{finetune} = \hat{\mathcal{L}}_{a \to v} + \hat{\mathcal{L}}_{v \to a}$ where $\hat{\mathcal{L}}_{v \to a}$ is the loss symmetric to $\hat{\mathcal{L}}_{a \to v}$.

Here all training refers to the finetune stage as we start from the sounding action checkpoints provided by Yang et al. [15] in their github. To keep settings close to the original paper's we used the same hyperparameters, outlined in table 1.

Learning Rate	5e-5
Batch Size	32
Temperature	0.2
Sample Rate	16 kHz
Visual Encoder Slots	7
Optimizer	Adam

Table 1: Hyperparameters used in training on synthetic and real data.

3.4 Synthetic Dataset

This dataset consists of about ten thousand collisions making it about a tenth the size of Epic Kitchens [4]. However the synthetic dataset is less diverse as there are only 11 varieties of objects to generate with, limiting material and visual diversity. In each clip there are 6 objects to ensure a diversity of collision combinations while keeping the scene simple enough for the model to learn from it. Each clip is 1.5s, the ideal length as found in [2]. It should be noted that the timing can change if the collision happens less than 0.5s before the end of the video or less than 1s after the start of the video. These numbers can be found in table 2

Sample Count	10616
Video Length	5s
Clip Length	1.5s
Length Before Collision	1s
Length After Collision	0.5s
Objects per Clip	6
Object Variations	11
Validation Split	0.2
Error rate	23%

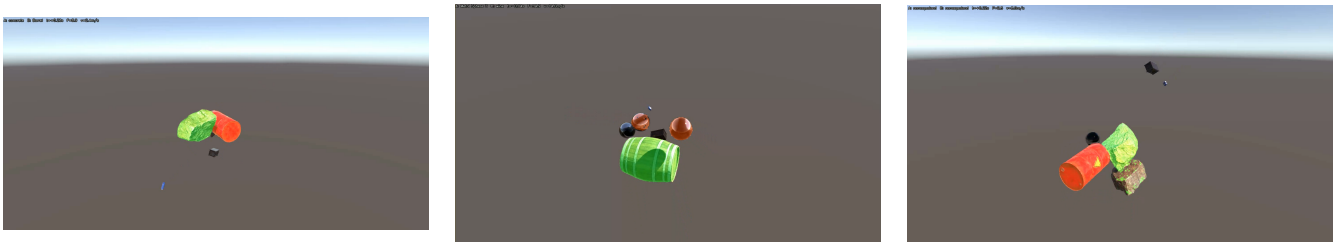
Table 2: Statistics of the synthetic dataset generated and experimented on.

Additionally, to ensure the quality of our labels, we manually reviewed a hundred samples and found that for the subset we label (those with both object centers and collision points visible) about 23% of generated samples have major issues. Some of the masks identify the wrong objects, sometimes the point prompt can visually overlap with an object leading to a mislabeling, thankfully since all objects collide in the middle, the chaos makes the two labeled objects collide often. This means the collision will be at a different frame of the video clip than expected but should not prevent training. While a 23% error rate may sound too high to train on, the training methods introduced by Yang et al. [15] and Chen et al. [2] are weakly supervised. This means that some level of label noise is expected and the model should then be relatively robust.

The more fatal mistake of the labeling pipeline currently is its tendency to make masks bleed to objects on contact or that look in contact. While this mostly leads to some likely unproblematic artifacts, the mask bleed also causes worse mistakes on collisions. This contact often makes the

masks bleed and distort after contact, limiting the usefulness of the end of the clips. Thankfully this problem is somewhat mitigated by using a shorter clip window after the collision than before. However the performance difference to training with the real dataset despite these labeling limitations can only be known after experimenting with the model.

Some examples of labeled frames can be seen in figure 4. It can be seen that there are some errors though not all make the masks unusable.



(a) (b) (c)

Figure 4: 3 examples of labeled clip frames. (a) is a correct example, here the model correctly segmented the boulder and barrel. (b) is an example of a bleed issue, the mask bled from one metal ball to the other. (c) is an example of a bleed and swap issue, the mask bleeds on a boulder and segments the wrong items. As the items still collide this is considered usable.

3.5 Control Dataset

To ensure results are comparable to previous papers, we compare performances on the Epic Kitchens dataset [4]. The Epic Kitchens dataset consists of 55 hours of first-person videos. The videos were filmed by 32 participants of different nationalities in their kitchens and without scripts. It is also notable that every video in this dataset was narrated by the cameraperson after the fact and segments were later labeled for the depicted action, adding the language modality used in previous parts of the model's training.

This dataset is particularly relevant here as it was used by Yang et al. [15] to train CCT. We use the masks they introduced, which were produced using an off-the-shelf hand-object interaction detection model and SAM2 for the training dataset, following the intuition that most object interactions involve or happen close to the cook's hands. This means some noise is to be expected from the training labels. On the other hand the validation dataset was hand labeled, making it quite reliable.

3.6 Hybrid Data Training

Two methods of adding this data are studied here. In the first method, the synthetic dataset is used as a warmup dataset. This method aims to quicken training on the main dataset. The intuition behind this method is that the scenes in the synthetic dataset are simpler than in the real dataset, there are fewer objects and visual clutter and there is no background noise. Those could make this simpler for the model to learn and act as a good pretraining before the model sees more complex

samples.

The second method is to add the synthetic data points to the main dataset and train on this combined data. The idea behind it is that training on two datasets may lead the model to simply "forget" what it could use on one of the datasets to instead learn representations that only apply to the second dataset it sees. Instead we make the model train on a composite dataset containing both real data and the synthetic data in order to force the model to learn a more generalizable method from training on more diverse data.

Both methods are compared to each other and the baseline over their performance on the real data from the Epic Kitchens validation set as real world performance improvements remain the end goal.

4 Experiments

Most of the experimental setup is shared across model training experiments. Training on purely synthetic datasets was run 10 times, because of the difference in training time models trained on datasets including real data these experiments are run 5 times. For the same reason and because performances stabilize quickly, purely synthetic training is run for 15 epochs and training on real data is run for 6 epochs. Training runs were allocated one A100 GPU. The model hyperparameters are shared across all models and are shown in table 1.

The experiments in this section aim to answer 3 questions:

- Is the synthetic dataset we generated coherent enough to be learned and close enough to the real data to translate into performance improvements on it?
- Can the use of synthetic-real hybrid datasets in training allow to improve performance over that of CCT?
- Which is the bottleneck at this dataset scale between data quantity and quality?

Because as mentioned previously, the focus here is identifying the objects involved in the interaction rather than exactly where they are the metric used in [15] and here is the top-1 accuracy. This puts more weight on sounding object identification rather than precise location. This is because this metric and inference method require the model to score the correct mask as most likely correct rather than make it segment each pixel in each frame by itself.

4.1 Synthetic Training

We must first ensure there is no catastrophic issue with the dataset that could make it impossible to learn or if it is too far from real data to generalize to it. To answer them we ran 10 repetitions of training, validating on both the synthetic and Epic Kitchens sets. First by studying the average loss curve of the models in figure 5 we can see that loss gets lower during training, though it slows after the first few steps. This shows that while there are some errors in the synthetic dataset it is still self coherent enough to optimize over. Although that does not necessarily mean it is of enough

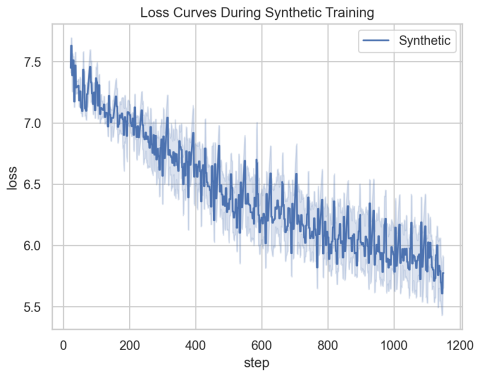


Figure 5: Learning curve of a sounding object detection model training on synthetic data. The loss starts around 7.5 and falls to around 7.0 in around 100 steps. After this, loss keeps slowly decreasing, eventually reaching about 5.5 at the end of training.

quality as this only means the model can learn the data it has seen in training. This also shows the robustness of the negative background prior method.

Figure 6 shows the average accuracies on each validation set. For the first 1-3 epochs of training, validation accuracy increases by an average of approximately 5 points and about 12 points on the synthetic validation set. Afterwards, performance greatly suffers and falls back to around their starting accuracies of about 38% on real data and 43% on synthetic data. Performance never really improves again but synthetic accuracy slowly improves back to around its peak accuracy of about 55%.

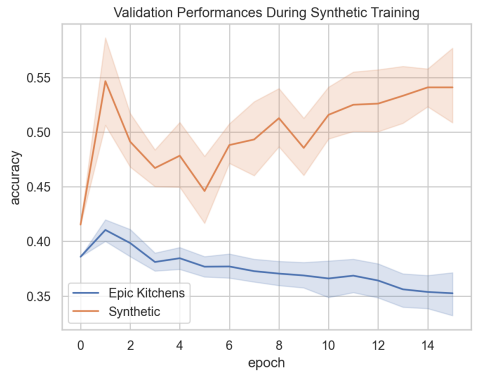


Figure 6: A line plot of the validation accuracy of 10 models trained on a synthetic Dataset. The accuracy on a synthetic and real validation set are plotted separately and start at 43% and 38% respectively. At first performance increases on both set. After 2-3 epochs performance collapses back to starting levels. Accuracy on the synthetic set slowly improves back to peak levels. Performance on the real data simply keeps worsening, making synthetic validation set accuracy higher all throughout.

This performance graph shows the model quickly overfits on the training dataset, which would explain the sudden accuracy decrease across both validation datasets. However the synthetic

validation set accuracy improves back to its peak while accuracy on the real dataset keeps falling. This suggests the model is able to learn some element of the simulation to predict masks that doesn't translate to real data. This underlines the importance of the sim2real gap for performance using synthetic data as a model with a more realistic environment would have less chances to learn a strategy not applicable to real data. This is more likely than the issue being object diversity because our baseline has relatively little diversity. Epic Kitchens was recorded by only 32 participants all in the same type of environment for every video, their kitchen, and thus with mostly similar objects (pots, pans, stoves, etc). Although the domain gap between kitchens and our set of objects (barrels, plastic bottles, boombox, ...) may be compounding with the sim2real gap to further decrease performance on this validation set.

4.2 Hybrid Data Training

To investigate the influence of the synthetic data on models trained using both datasets we ran 5 training repetitions of 3 models. A control model trained on Epic Kitchens, a model warmed on the synthetic, a model trained on the synthetic and Epic Kitchens training sets mixed into one. The peak accuracy reached by each method can be seen in figure 7. This shows that this dataset is unable to improve meaningfully on the CCT results.

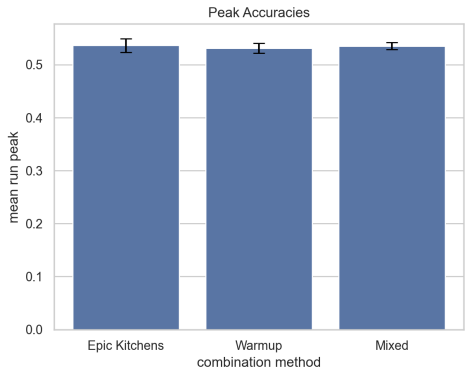


Figure 7: A bar plot of the peak accuracies achieved by models trained using three methods over 5 runs. One trained on epic kitchens, a second warmed on a synthetic dataset before epic kitchens and a third trained on a mix of synthetic and real dataset. All performances appear more or less equal at about 54% accuracy.

One reason it is difficult to make an impact on training is the speed of it as can be seen in figure 8. Peak accuracy is reached at the first epoch almost every time for every training set. As all the improvements happen so quickly the small change because of the synthetic datasets become too small to see or it may be that this dataset is not of sufficient quality to help a model above a certain level of performance. This could mean that the bulk of the improvements come from the background penalty from the finetune loss. This experiment does not allow to conclusively prove this is the right explanation over another. Exact performance numbers are shown in table 3.

These results also raise the question of why performance does not improve from adding the synthetic data with the real data over Epic Kitchens on its own. This could be because the amount of real data involved is so much larger than the amount of synthetic data (x100). It could also be

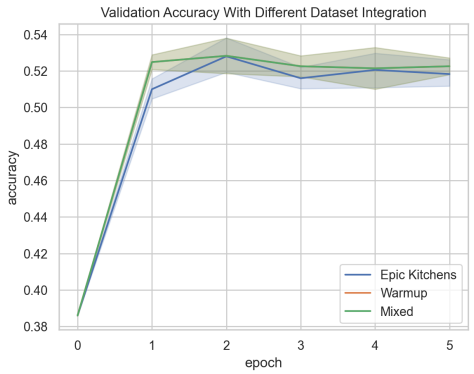


Figure 8: Line plot of the evolution of validation accuracy over training epochs of 3 models over 5 runs. Accuracies start around 39% and increase to about 53% where it stays stable for the rest of training.

Dataset	Peak EK Validation Accuracy
Epic Kitchens Control	0.536 ± 0.01
Warmed	0.531 ± 0.01
Mixed	0.535 ± 0.006

Table 3: Results of the hybrid data experiment, fine tuning the CCT model from its sounding actions checkpoints using datasets composed of both synthetic and real data.

because our synthetic data is less informative than real data because of the simpler scene (void vs a furnished kitchen), lower object diversity or another element that makes the synthetic data simpler and less informative.

4.3 Dataset Size Variations

To extrapolate how much bigger a similar dataset would need to be to improve performances more significantly we ran 10 runs of training on subsections of the full synthetic dataset. Models were trained on subsets of 50% and 75% for 10 runs. Their peak validation accuracies, the highest validation accuracy reached during training, on each dataset are compared to results of the full synthetic datasets runs in figure 9. Note these per-run peaks sit above the 55% peaks of the averaged curve in Figure 6, since individual runs reach their maximum at different epochs. Exact performance figures can be found in table 4.

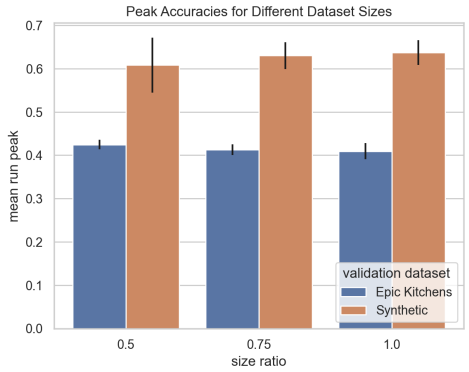


Figure 9: A bar plot of the peak accuracies reached on two validation sets reached by models during training on different fractions (0.5, 0.75 and 1) of a synthetic dataset. Performance on a given validation set seems constant with dataset ratio around 42% on Epic Kitchens and around 63% for the synthetic set. However standard deviation on the synthetic dataset seems larger for the smaller dataset.

Dataset	Peak Synthetic Validation Accuracy	Peak EK Validation Accuracy
Epic Kitchens Control	–	0.536 ± 0.01
Synthetic Only	0.637 ± 0.028	0.425 ± 0.01
Synthetic 0.5	0.608 ± 0.06	0.424 ± 0.01
Synthetic 0.75	0.630 ± 0.03	0.413 ± 0.01

Table 4: Results of experiment on synthetic dataset size to fine tune the CCT model from its sounding actions checkpoints.

It can be seen that even halving the number of samples in the synthetic dataset barely affects validation accuracy. This reinforces findings in subsection 4.1 that peak performance is more dependent on sample quality (size of the sim2real gap, labeling quality, etc.) than quantity here.

However the larger standard deviations for smaller training sets on the synthetic validation sets suggests that, at least at these scales, data quantity improves training stability but not performance. Though the reason why dataset size does not affect standard deviation on the real data validation set remains an open question, a few possibilities stand out. It may be that the model is able to extract all the information it can from the synthetic dataset with fewer samples than tested here but less reliably, indicating a lack of data diversity or too much noise in the data. This could also support the possible conclusion from the hybrid training experiment that most of the performance improvements are coming from the object-centric prior of the finetuning stage rather than the amount of data. Meaning the dataset's size only stabilizes training without changing outcomes meaningfully at the scale we study here.

5 Conclusions and Further Research

In this work we introduced a synthetic data generation and labeling pipeline for collision videos and asked whether the data it produces can support sounding object detection. The generated data is self-coherent enough to be learned from and transfers partially to real footage: training on synthetic data alone briefly raises real-validation accuracy before the model overfits to simulation-specific cues. When real data is available, however, neither warming up on synthetic data nor mixing it in improves over Epic Kitchens alone, and varying the synthetic set between 50% and 100% of its maximum size of 10,000 samples leaves peak accuracy essentially unchanged.

We must be cautious in reading these results. The size experiment shows that, within the range we were able to generate ($\sim$10,000 samples), adding synthetic quantity does not raise peak performance; it does not establish that quantity is irrelevant in general, and a dataset closer to Epic Kitchens in scale might behave differently. What the experiments support more directly is that the useful signal in our synthetic data is bounded by its quality (labeling accuracy, size of the sim2real gap) rather than by its amount at these scales. We did not manipulate quality directly, so this remains an inference from the size and hybrid experiments rather than a controlled result.

This work has four main limitations. The first is the labeling error rate, which lowers data quality and is at least partly responsible for the limited improvements we observe. The second is that our simulation features only 11 object varieties, and this lack of diversity likely hurts performance. The third is that our deliberately simple simulation engine and scene leave the sim2real gap probably large relative to state-of-the-art simulators, which would, however, be slower to run. Finally, peak accuracy is an optimistic metric, especially under noisy training.

More broadly, our results give a tempered picture of synthetic data for sounding object detection. A lightweight, physics-grounded pipeline can cheaply produce data a model can learn from, but in its current form that data neither replaces nor meaningfully augments real egocentric footage. The limiting factor is currently fidelity rather than volume, which suggests the productive path is more realistic simulation and more reliable labels rather than simply more clips. Read this way, these results are in and of themselves informative of where future effort is best spent.

Concretely, three avenues stand out. Greatly increasing dataset size, to a scale comparable with Epic Kitchens, would test whether quantity matters beyond the range we could study. More robust sample labeling would raise data quality directly. And a higher-fidelity simulation engine would narrow the sim2real gap. The latter two target the bottleneck our experiments point to and, at the scales studied here, look the most promising.

References

[1] Steve Borkman, Adam Crespi, Saurav Dhakad, Sujoy Ganguly, Jonathan Hogins, You-Cyuan Jhang, Mohsen Kamalzadeh, Bowen Li, Steven Leal, Pete Parisi, Cesar Romero, Wesley Smith, Alex Thaman, Samuel Warren, and Nupur Yadav. Unity perception: Generate synthetic data for computer vision. *CoRR*, abs/2107.04259, 2021.

[2] Changan Chen, Kumar Ashutosh, Rohit Girdhar, David Harwath, and Kristen Grauman. Soundingactions: Learning how actions sound from narrated egocentric videos, 2024.

[3] Manuel Cherep and Nikhil Singh. Contrastive learning from synthetic audio doppelgangers, 2025.

[4] Dima Damen, Hazel Doughty, Giovanni Maria Farinella, Sanja Fidler, Antonino Furnari, Evangelos Kazakos, Davide Moltisanti, Jonathan Munro, Toby Perrett, Will Price, and Michael Wray. The epic-kitchens dataset: Collection, challenges and baselines, 2020.

[5] Benjamin Elizalde, Soham Deshmukh, Mahmoud Al Ismail, and Huaming Wang. Clap: Learning audio concepts from natural language supervision, 2022.

[6] Chuang Gan, Jeremy Schwartz, Seth Alter, Martin Schrimpf, James Traer, Julian De Freitas, Jonas Kubilius, Abhishek Bhandwaldar, Nick Haber, Megumi Sano, Kuno Kim, Elias Wang, Damian Mrowca, Michael Lingelbach, Aidan Curtis, Kevin T. Feigelis, Daniel M. Bear, Dan Gutfreund, David D. Cox, James J. DiCarlo, Josh H. McDermott, Joshua B. Tenenbaum, and Daniel L. K. Yamins. Threedworld: A platform for interactive multi-modal physical simulation. *CoRR*, abs/2007.04954, 2020.

[7] Arthur Juliani, Vincent-Pierre Berges, Ervin Teng, Andrew Cohen, Jonathan Harper, Chris Elion, Chris Goy, Yuan Gao, Hunter Henry, Marwan Mattar, and Danny Lange. Unity: A general platform for intelligent agents, 2020.

[8] Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. Object-centric learning with slot attention. *CoRR*, abs/2006.15055, 2020.

[9] Shentong Mo and Pedro Morgado. Localizing visual sounds the easy way, 2022.

[10] Kranti Kumar Parida, Omar Emara, Hazel Doughty, and Dima Damen. Segmenting collision sound sources in egocentric videos, 2025.

[11] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024.

[12] Arda Senocak, Tae-Hyun Oh, Junsik Kim, Ming-Hsuan Yang, and In So Kweon. Learning to localize sound source in visual scenes, 2018.

[13] James Traer, Maddie Cusimano, and Josh H McDermott. A perceptually inspired generative model of rigid-body contact sounds. In *Digital Audio Effects (DAFx)*, volume 1, page 3, 2019.

[14] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding, 2019.

[15] Mengyu Yang, Yiming Chen, Haozheng Pei, Siddhant Agarwal, Arun Balajee Vasudevan, and James Hays. Clink! chop! thud! – learning object sounds from real-world interactions, 2025.